DOCUMENTACIÓN TÉCNICA

SISTEMA DE ASISTENCIAS FACIAL

Flujo principal de la web y reconocimiento facial

Cómo interactúa el guardia con el panel, cómo se procesa una captura y cómo se registra cada entrada o salida.

Resumen ejecutivo

La aplicación es un panel local construido con Flask. El navegador solicita acceso a la cámara, envía una captura al backend y el módulo de reconocimiento la compara contra las imágenes del directorio de empleados. Cuando hay una única coincidencia válida, el sistema registra el evento en el CSV existente.

EN UNA MIRADA

1. **Ingresar**
   Autenticación por sesión antes de acceder al panel.
2. **Capturar**
   La cámara se habilita con una acción explícita.
3. **Reconocer**
   Se detecta, codifica y compara el rostro.
4. **Registrar**
   Se escribe entrada o salida y se informa el resultado.

Alcance actual: prototipo local y académico. Usa una contraseña compartida hardcodeada, CSV y filesystem; no incluye usuarios individuales, roles, detección de vida ni una base de datos.

01 / INTERACCIÓN

Flujo principal de la web

El flujo separa la experiencia del navegador de la lógica de reconocimiento. La cámara solo participa en la pantalla de asistencia; la administración de empleados y la consulta de registros siguen protegidas por la sesión.

1 Login y sesión

El usuario envía la contraseña a `/login`. Flask compara el valor y guarda `authenticated` en la sesión.

↓

2 Pantalla Asistencia

El navegador muestra el video. JavaScript pide permiso mediante `getUserMedia` solo al pulsar "Iniciar cámara".

↓

3 Captura JPEG

El frame del video se dibuja en un canvas y se envía como `multipart/form-data` junto con `event_type`.

↓

4 Endpoint Flask

`POST /api/attendance/recognize` valida tipo, tamaño y formato; luego llama al reconocedor facial.

↓

5 Resultado visible

La UI informa registro, duplicado, rostro desconocido, ausencia de rostro, múltiples rostros o error.

Rutas principales

RUTA	FUNCIÓN	PROTECCIÓN
`/`	Pantalla de cámara para marcar asistencia.	Sesión requerida
`/attendance`	Consulta el CSV y filtra por empleado o tipo.	Sesión requerida
`/employees`	Galería, carga y eliminación de imágenes.	Sesión requerida
`/api/attendance/recognize`	Recibe la captura y devuelve JSON.	Sesión requerida
`/employees/images/<filename>`	Entrega imágenes válidas del padrón.	Sesión requerida

Consulta de registros: la vista `/attendance` lee el CSV con la cabecera existente, muestra las filas ordenadas por fecha y hora, y no modifica el archivo.

02 / RECONOCIMIENTO

Cómo funciona el reconocimiento facial

El backend convierte la imagen en datos comparables. No compara nombres ni archivos: compara codificaciones faciales y usa una distancia matemática junto con una tolerancia.

A	**Preparar el padrón**	`cargar_empleados` recorre las imágenes válidas de `assets/empleados/`. Cada nombre produce un ID estable y cada foto se valida con exactamente un rostro.
B	**Detectar rostros**	`face_locations` localiza las caras del frame. El modelo configurado es `hog`, adecuado para uso local en CPU.
C	**Codificar rostros**	`face_encodings` transforma cada rostro detectado en una representación numérica que permite comparar identidades.
D	**Calcular distancia**	`face_distance` calcula la distancia entre la captura y cada empleado. Se elige el empleado con la menor distancia.
E	**Aplicar tolerancia**	`compare_faces` decide si la mejor coincidencia alcanza la tolerancia configurada, actualmente `0.6`. Una distancia menor suele indicar mayor similitud.
F	**Convertir a resultado**	El sistema devuelve un objeto `Reconocimiento` con coincidencia, ID, nombre, ubicación y distancia para que Flask tome la decisión final.

Escala 0.25

El frame se reduce antes de detectar para mejorar el rendimiento.

Un rostro

La captura válida debe contener exactamente una cara.

Modelo HOG

Detección local sin depender de un servicio externo.

Decisiones del endpoint

registered

Una persona coincide y se agrega la marcación al CSV.

duplicate

Ya existe el mismo empleado, fecha y tipo de evento.

no_face / multiple_faces

La captura no es válida para una marcación segura.

unknown / error

No coincide con el padrón o falló la lectura/procesamiento.

> **Importante:** el reconocimiento facial no equivale a detección de vida. Una fotografía podría ser aceptada si supera la comparación; para operación real habría que sumar liveness, controles de acceso y auditoría.

03 / ARQUITECTURA

Librerías y responsabilidades

Cada librería tiene una responsabilidad concreta. Flask coordina el flujo web; OpenCV y NumPy preparan la imagen; face_recognition realiza la detección y comparación.

LIBRERÍA	USO EN EL SISTEMA	PUNTO DE ENTRADA
Flask 3.1	Rutas, sesiones, respuestas JSON, templates Jinja y servidor local.	`asistencia_facial/web.py`
face_recognition 1.3.0	Carga de imágenes, detección de ubicaciones, codificaciones, distancias y decisión de coincidencia. Usa dlib por debajo.	`recognition.py`
OpenCV	Decodifica el JPEG recibido, redimensiona frames y convierte BGR a RGB.	`_decode_frame` y `reconocer_frame`
NumPy	Representa la imagen como array y permite buscar la distancia mínima con `argmin`.	`recognition.py`
Jinja2	Renderiza login, asistencia, empleados y la tabla de registros.	`templates/*.html`
Werkzeug	Normaliza nombres de archivos y ayuda a mantener las cargas dentro del directorio permitido.	`EmployeeCatalog`
csv + pathlib	Lee y escribe el registro, maneja rutas y mantiene la persistencia simple en filesystem.	`attendance.py`

Persistencia de asistencia

El CSV usa esta cabecera:

`id_empleado,nombre,fecha,hora,tipo,distancia`

- La escritura se centraliza en `registrar_evento`.
- La deduplicación usa empleado + fecha + tipo.
- Se permite una entrada y una salida por día.
- La distancia se conserva para revisión posterior.

Directorio de empleados

Las imágenes viven en `assets/empleados/`. Al cargarlas:

- Se valida extensión y nombre.
- Se exige exactamente un rostro.
- Se calcula la codificación en memoria.
- Eliminar la foto no borra el historial CSV.

Referencias directas al código

`web.py:266-301`
Vista protegida y filtros de registros.

`web.py:343-390`
Reconocimiento y registro del evento.

`recognition.py:38-62`
Validación y codificación de una imagen.

`recognition.py:138-188`
Detección y comparación del frame.

`attendance.py:9-16`
Esquema estable del CSV.

`attendance.py:37-88`
Escritura y deduplicación.